

Mirage: An Adaptive Cyber Deception and Active Response Platform

Project Proposal

Group Members:

M Amar Waqar	23L-3035
Hussain Nawaz	23L-3018
Haseeb Ahmad	23L-3069

National University Of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

Abstract

Most network defence today works by watching logs and matching known attack signatures. This creates a flood of alerts, many of them false alarms, and usually spots an intruder long after they have already broken in. Mirage takes the opposite approach. Instead of looking for the attacker in a sea of normal traffic, it sets up fake targets that no real user would ever touch. The platform runs decoy services in Docker containers (a fake admin panel, a fake internal SSH server and a fake AWS S3 storage endpoint) and spreads realistic fake secrets across the environment, such as fake API keys, database passwords, fake database records and canary documents. These fake items are arranged as a trail of breadcrumbs that leads an intruder deeper into the trap. Any time someone touches a decoy or a fake secret, Mirage raises a reliable alert, labels it with the matching MITRE ATT&CK technique and MITRE Engage activity, and passes it to a response controller. The controller can watch the attacker, slow them down, or shut them out by blocking their address at the firewall and ending their session, while saving the evidence. A live dashboard shows the attacker scanning, stealing credentials and moving through the trap, along with every automatic response. Mirage will be tested in an isolated lab with scripted attacks, measuring how fast it detects and contains an intruder, how few false alarms it raises, and how many attack techniques it catches.

1. Introduction

Once an attacker gets past the outer defences of a network, they tend to move around quietly, stealing passwords, finding services and copying data out. Reports show that these break-ins often go unnoticed for weeks. The usual defences, such as firewalls, signature based intrusion detection and log analysis tools, are mostly reactive. They have to pick out the few harmful actions from a huge amount of normal activity, which leads to tired analysts and missed attacks.

Cyber deception works differently. It fills the network with decoys (fake systems and services) and honeytokens (fake passwords, keys, files and records) placed where an attacker is likely to look. Because these items have no real purpose, anyone who touches them is almost certainly an intruder, so the alerts are accurate and there are very few false alarms. The MITRE Engage framework describes this way of working as engaging the attacker, with goals such as Expose, Affect and Elicit and methods such as lures and decoy files. Paid products that do this already exist, but they are expensive and closed. Free honeypots exist too, but they are usually single traps that only record what happens and never react.

Mirage sets out to fill this gap with one open platform that brings together the decoys, a tool that creates and tracks the fake secrets, an automatic detection and response controller, and a live dashboard. Its main ideas are: (i) a planned breadcrumb trail, so the platform always knows how far an intruder has gone; (ii) fake secrets that are each unique, so every alert shows exactly which item was stolen and from where, and so activity from different addresses using the same stolen password can be tied back to one attacker; and (iii) a range of automatic responses, from quietly watching to fully shutting the attacker out. On top of this, the fake SSH server is powered by a large language model that acts like a real Linux shell, so attackers stay longer and show more of what they are trying to do.

2. Goals and Objectives

The main goal of Mirage is to show that an automatic deception platform can spot an attacker quickly after a break-in, raise almost no false alarms, and shut the attacker out on its own. The specific objectives are:

- To build and run decoy services in Docker containers: a fake admin web panel, a fake internal SSH server, and a fake AWS S3 storage endpoint.
- To build a tool that creates realistic, uniquely tagged fake secrets (fake API keys, database passwords, fake database records, canary documents such as passwords.xlsx, and fake links) and spreads them across the environment.
- To give the fake SSH server an AI driven shell that uses a large language model to produce realistic, consistent command output, so attackers stay longer, and to label every command with its ATT&CK technique.
- To link the decoys and fake secrets into a breadcrumb trail that guides an intruder along a planned path and shows how far they have reached.
- To build a detection engine that checks incoming traffic against the real service routes, raises an instant alert the moment a decoy or fake secret is touched, and labels each event with its MITRE ATT&CK technique and MITRE Engage activity.
- To fingerprint attackers and their tools (such as SSH client fingerprints, HTTP header patterns and scanner signatures) and tie together sessions that reuse the same stolen password from different addresses.
- To build a response controller that can watch, slow down, or shut out an attacker, including blocking their address at the firewall for a set time, ending their session, and saving a snapshot of the affected decoy.
- To build a live dashboard that shows attacker scanning, credential theft, progress through the trail, session replays and the automatic responses taken.
- To test the platform against scripted attacks and normal traffic, reporting how fast it detects and contains an attacker, its false alarm rate, and how many ATT&CK techniques it catches.

3. Scope of the Project

Mirage will be built and shown entirely inside an isolated Docker based lab network. It has the following parts.

3.1 Decoy Infrastructure

Docker containers running decoy services that are left exposed on purpose. Each one is filled with realistic made up content (staff names, file histories and logs) so it looks like a real, lived in system:

- **Fake admin panel:** a convincing internal login and admin page with fake users, settings and planted passwords. Every login attempt, password guessing run, folder scan and injection attempt is recorded.
- **Fake internal SSH server:** a working SSH login with a realistic file system that hides planted secrets (such as ~/.aws/credentials, .bash_history and config files). Any command it does not script directly is handled by the AI driven shell described below, and whole sessions are recorded so they can be replayed.
- **Fake AWS S3 endpoint:** an S3 style storage service (built on MinIO or LocalStack) that accepts the planted fake AWS keys, so we can watch exactly what an attacker does with stolen cloud keys.

3.2 AI Driven Interactive Shell

Ordinary SSH honeypots only know a fixed list of commands, so skilled attackers spot them quickly and leave. Mirage's fake SSH server mixes two layers:

- **Scripted layer:** common commands and all the planted breadcrumbs (such as ~/.aws/credentials, .bash_history and config files) come from a fixed fake file system, so the bait is always there and always the same.
- **AI layer:** any other command is passed to a large language model that is told to act as one specific fake server (with a set hostname, operating system, services and users) and produces realistic output for commands such as ps, netstat, config files or database clients.
- **Session memory:** anything the attacker changes (new files, folders or downloaded tools) is remembered and fed back to the model, so the answers stay consistent for the whole session.
- **Command labelling:** every command is matched to an ATT&CK technique (such as discovery, credential access, tool download or persistence) and shown live on the dashboard, raising the attacker's risk score.

This keeps the attacker busy for longer and captures more of what they do, which supports the MITRE Engage goal of drawing out attacker behaviour.

3.3 Honeytoken Fabric

A tool that creates realistic fake secrets, gives each one a unique tag and records where it was placed, then spreads them across the environment:

- Fake API keys and AWS keys in real looking formats, placed in .env files, config files and git history.
- Fake database passwords and honey records (fake rows in a database) that raise an alert whenever they are read or copied.
- Canary documents (such as passwords.xlsx and internal .docx and .pdf files) that quietly call home when opened.
- Fake links and DNS canaries (such as entries in robots.txt and unique subdomains) that reveal scanning.

These items are joined into a **breadcrumb trail**, for example: robots.txt → exposed backup .env file → SSH login → fake SSH server → AWS keys → fake S3 bucket → canary document. Because every item is unique, each alert shows exactly which one was stolen, where it came from, and how far along the trail the attacker has reached.

3.4 Detection and Active Response Controller

- **Route checking:** a reverse proxy checks incoming traffic against the real service routes. Any request to a decoy route, or any use of a fake secret, is treated as hostile.
- **Instant alerts:** every time a decoy or fake secret is touched, Mirage raises an alert that includes the attacker's address, their fingerprint, the item touched, and the matching MITRE ATT&CK technique and MITRE Engage activity. Alerts are sent to the dashboard and to channels such as email, Discord or Slack.
- **Attacker profiling:** each attacker gets a risk score built from their fingerprint, the tools they use and how far they have come along the trail. Sessions from different addresses that reuse the same stolen password are tied to one attacker.
- **Response playbooks:** a policy engine picks the response. Watch lets low risk activity continue so Mirage can learn more. Slow down drags the attacker's connection out. Shut out blocks the attacker's address at the firewall (using nftables and ipset, for a set time) and ends their session. Save then shut out first takes a snapshot of the decoy for later study. An allow list makes sure the defenders are never locked out by mistake.

3.5 Containment and Disruption

Once an attacker's risk score crosses the limit, Mirage runs a disruption sequence. Every step targets only the attacker's own session and the decoy containers that Mirage owns, so the whole response stays inside the isolated lab and never reaches any outside machine. The aim is to stop the break-in on the spot, waste the attacker's time, and keep the evidence:

- **Session freeze:** the attacker's connection is slowed to a stop, so their tools stall, hang and time out.
- **Resource overload:** the attacker's session on the decoy is pushed to full load until their terminal locks up and stops responding.
- **Download choke:** if the attacker tries to copy out a planted database dump, Mirage hands back an oversized decoy file that clogs their own download and kills the theft.
- **Lockout and notice:** the session is ended, a message tells the intruder they have been caught and shut out, and their address is blocked at the firewall for a set time.
- **Evidence capture:** before the decoy is reset, Mirage saves a snapshot of the container and the full session recording so the break-in can be studied later.

On the dashboard this all happens live. The attacker's progress along the trail stops dead, each disruption step lights up one after another, and the response log fills in real time while the intruder is frozen out.

3.6 Command and Telemetry Dashboard

A live web dashboard that shows a map of the lab with attacker to decoy activity, a timeline of how far each attacker has come, credential theft and fake secret events, replays of recorded SSH sessions, the ATT&CK techniques seen, and a log of the automatic responses, with the option for an analyst to release or step up a response by hand.

3.7 Evaluation

The platform will be tested in the lab with an attacker container (such as Kali Linux) running scripted attacks using common tools like nmap, gobuster, hydra and the AWS CLI, alongside a traffic generator that acts like normal users. We will measure the average time to detect and time to contain an attacker, the false alarm rate, the number of ATT&CK techniques caught, and how much longer attackers stay in the AI shell compared with an ordinary fixed command honeypot.

3.8 Out of Scope

Everything runs inside the isolated lab. Mirage never attacks the intruder's own machine, and it uses only fake cloud accounts and fake passwords, never real ones.

4. Initial Study and Work Done so Far

We have studied the MITRE Engage framework for engaging attackers and the MITRE ATT&CK knowledge base, which we will use to label what attackers do. We read the early work on honeypots and honeytokens, including Spitzner's writing on both and the research on honeyfiles as fake files for catching intruders, along with a survey of deception technology.

We also looked at existing free tools. Cowrie is a widely used fake SSH and Telnet server. Thinkst Canarytokens hands out trackable items such as fake AWS keys and canary documents. T-Pot bundles many honeypots behind one dashboard. Newer projects such as shelLM and Galah show that large language models can make honeypots far more believable. We found that these tools mostly work on their own and only watch: they record what happens but do not join the decoys into a single trail, tie stolen passwords back to one attacker, or react on their own. Mirage is built to close these gaps.

From this study we have settled on the overall design and its main parts, planned the breadcrumb trail, and chosen the tools: Docker Compose for the lab, Python with FastAPI for the detection and response controller, a Paramiko based fake SSH server with a language model shell, MinIO for the S3 endpoint, nftables and ipset for blocking attackers, Redis for passing events around, and a React dashboard that updates live over WebSockets. For fingerprinting we will build on known methods such as HASSH for SSH clients and JA3 for TLS clients.

References

[1] The MITRE Corporation, "MITRE Engage: A framework for adversary engagement," 2022. [Online]. Available: https://engage.mitre.org. [Accessed: Oct. 24, 2026].

[2] The MITRE Corporation, "MITRE ATT&CK: Adversarial tactics, techniques, and common knowledge." [Online]. Available: https://attack.mitre.org. [Accessed: Oct. 24, 2026].

[3] L. Spitzner, Honeypots: Tracking Hackers. Boston, MA: Addison-Wesley, 2002.

[4] L. Spitzner, "Honeytokens: The other honeypot," SecurityFocus, Jul. 2003. [Online]. Available: https://www.securityfocus.com/infocus/1713.

[5] J. Yuill, M. Zappe, D. Denning, and F. Feer, "Honeyfiles: Deceptive files for intrusion detection," in Proc. 5th Annu. IEEE SMC Information Assurance Workshop, West Point, NY, 2004, pp. 116–122.

[6] D. Fraunholz et al., "Demystifying deception technology: A survey," arXiv preprint arXiv:1804.06196, 2018.

[7] M. Sladić, V. Valeros, C. Catania, and S. Garcia, "LLM in the shell: Generative honeypots," in Proc. IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), 2024.

[8] Cowrie Project, “Cowrie SSH/Telnet honeypot.” [Online]. Available: https://github.com/cowrie/cowrie. [Accessed: Oct. 24, 2026].

[9] Thinkst Applied Research, “Canarytokens.” [Online]. Available: https://github.com/thinkst/canarytokens. [Accessed: Oct. 24, 2026].

[10] Deutsche Telekom Security, “T-Pot: The all-in-one honeypot platform.” [Online]. Available: https://github.com/telekom-security/tpotce. [Accessed: Oct. 24, 2026].

[11] A. Karimi, “Galah: An LLM-powered web honeypot.” [Online]. Available: https://github.com/0x4D31/galah. [Accessed: Oct. 24, 2026].

[12] Salesforce, “HASSH: A profiling method for SSH clients and servers” and “JA3: A method for profiling SSL/TLS clients.” [Online]. Available: https://github.com/salesforce/hassh, https://github.com/salesforce/ja3. [Accessed: Oct. 24, 2026].